

小芽：宠物对战游戏里的 AI Coach

设计与实现说明

周宇宸 腾讯 IEG 面试题提交稿 2026 年 9 月

本项目开发过程中使用 AI agent 辅助代码实现、测试与文档整理；需求理解、产品设计、系统架构、角色边界、判断规则、实验设计与最终验收由本人负责。

我做了一个能直接玩起来的 demo，并内置了「小芽」AI Coach，涵盖军师、陪练、老师三种角色感。游戏里的确定事实——伤害怎么算、谁先出手、哪些行动合法——全部由规则和结算器负责。

所以它不只是聊天机器人：模型不自己算伤害、不自己决定哪些行动合法、不编造历史，也不猜对手还没发生的那一手。固定规则能回答结构化的提问，但玩家可以用任意方式开口。一句「我是不是老舍不得换宠？」，要接住它得先读懂这是在问跨局的行为习惯，而不是某个伤害数字，再去翻跨局记录、当场数一遍换宠次数，最后把查到的几件事讲成一句解释。模型的价值就在这一步：把开放式语言映射到游戏状态、历史记录和知识查询上，再基于核验过的结果组织解释。

三种角色各解决一个问题：军师摆出这一回合的取舍并给出可查的依据，老师只用当时看得见的信息复盘讲解，陪练记得住、有情绪但不越界。为了让这三套标准同时成立，合法行动先由结算器算完；模型拿到一个受控的只读工具循环，最多调三次，工具名、参数、重复调用、次数与回执大小逐次由程序校验；回执与生成结果对不上就丢掉模型那句，退回本机结论。

最大的限制是没有真人数据：学习效果只做过离线实验和自动检查，不能说明有人真的学会了。

谁负责什么

规则引擎：计算伤害、合法行动、速度与结算结果

知识检索：查询规则与战术知识

记忆模块：保存对局事件、明确偏好和教学记录

LLM：理解玩家问题、判断需要查什么、基于核验结果组织解释

1 题目与取舍

这道题要求给一个宠物对战游戏做一个 AI Coach，要有军师、陪练、老师三种角色感，参考方向列了 Agentic RL、RAG、ReAct、Memory Stream / Reflection，产品参考是和平精英的「小田」和王者荣耀的「灵宝」。我的理解是：要交付的是 AI Coach

本身，游戏只是让 AI Coach 有局面可看的那个环境；三种角色感也不是三种说话语气，而是三套不同的正确性标准——军师要算得准，还要说得出依据；老师只允许用当时看得见的信息讲话；陪练要记得住，而且不越界。

第一个动作是划边界，不是写代码。有两件事我一开始就决定不做。一是**美术与完整游戏内容**：规模控制在 14 只宠物、7 种属性、5 个关卡，这个量已经足够产生真实的取舍：先抢着收尾还是留能量，换一只上来承伤还是原地吃药。二是**联网 PVP**：本地对战是同机分屏，双方各自锁定行动，然后一起结算；服务端只接收浏览器传来的状态快照，不掌握权威对局。这条差距是架构问题，不是工程量问题。

三种角色我全部做完了，题目只要求闭环其中一种。它们的难点并不重叠：军师的难处在分工，哪些判断交给引擎、哪些话交给模型；老师的难处在时间口径，复盘只能用回合开始前看得见的东西；陪练的难处在克制，既不越界去当军师，也不评价玩家。只做一种，另外两类的毛病不会暴露出来。

2 交付了什么

2.1 三种角色共用的机制，以及接进来的四项技术

军师、陪练、老师是三种角色，共用同一套游戏状态、记忆、规则、门控和回答校验机制，但根据场景走不同的回答路径。需要开放式理解和解释时才调用模型；能由本地记录直接确定的内容则直接生成。玩家提问、或代码判定该开口时，请求先过门控，再由有界的工具循环去查证，查到的回执必须过校验，最后才允许输出——或者允许沉默。图 1 是这七步各自的出处。



图 1 AI Coach 共用的状态、查证与输出机制，以及七个环节各自的出处

工具循环是按这样的分工跑的：明确需要查证的问题，第一个必须调用的工具由程序直接指定；一部分默认参数也由程序构造，例如没点名回合时取最近一个已结算的回合、没点名行动时从玩家那句话里解析候选。已经有回执但仍然不够，模型可以在允许的工具集合里继续选工具和参数。工具名、参数、重复调用、调用次数和回执大小全部由程序校验，最多 3 次受控的只读工具调用；寒暄这类问题可以完全不调工具。经过模型生成的回答都要过同一道回答校验：和回执对不上、或者说得过长，就丢掉模型那句，改用本机已核验的结论。

这些回答走的是不同的路，不是所有话都经过模型。最常见的一条是本地规则直接生成：局面数据本来就齐全，本机模板把结论拼出来，不调模型也能完整游玩。第二条是模型路径：有工具可查时先走上面那圈查证，再由模型写正文。第三条是陪练主动开的那一句，由已记录的事件读数直接生成，全程不经过模型。第四条是兜底：模型没接上、超时、说得过长，或和回执对不上，就丢掉模型那句，退回本机结论，并把原因写在界面上。

题目点名的四项技术里，前三项落在这套机制的不同位置上；第四项 Agentic RL 我不做成系统的一部分，只按两个离线策略实验来探索。每一项都只做到它的一部分，下面把做到哪一步、没做到什么写清楚。

ReAct：受控的工具调用。对应共用机制里的第 3、4 步。模型发现手上依据不够时可以继续查游戏状态、规则、历史回合或模拟结果。该不该查由代码判定：该查的时候，第一个必须调用的工具由程序直接指定，一部分默认参数也由程序构造。剩下还差什么，模型在允许的工具里自己选，但工具名、参数、重复调用、调用次数和回执大小逐次由程序校验，最多 3 次只读的受控工具调用，最终只生成一次正文。这里没有做模型的思维链，也没有让它自由决定调用哪些工具。要如实说的是：`reports/live-model-eval.json` 里 44 条真实模型记录的受控工具调用次数分布是「0 次 36 条、1 次 8 条」，真实评测中一次多步工具链都没有观察到；有界循环本身靠确定性测试覆盖。

RAG：只查知识，不查局面。检索的是本地规则与战术知识，共 93 条卡。词项检索先出候选，装了本地多语言 MiniLM 时再出语义候选，两路结果按名次融合；语义服务不在就退回纯词项检索。它不算当前局面的事实：血量、伤害、合法行动由引擎当场算，检索只回答「这条规则怎么说」。

Memory Stream / Reflection：结构化记忆，不是自由总结。当前实现保存结构化的对局事件、明确偏好和教学记录，并根据多次独立行为决定要不要减少提示、要不要重教。同一类独立行动攒到 3 条，才形成一条带依据和证据 ID 的记录；证据不够就明说「继续观察」，新的反证出现时这条判断会被推翻。没有让 LLM 自由总结记忆，也没有实现完整的 Memory Stream。

额外探索：两个离线策略实验，不在线上。题目方向里点到的 Agentic RL，在这里只落到两个离线策略实验上，都不是系统主体，也不在线上。一个是干预时机：用 Q-learning 学「什么时候值得主动提醒」，提示次数从每局 5.89 次降到 1.7 次，但提示减少的同时帮助量也下降了，从 1.38 次降到 0.86 次。一个是小模型工具选择：SmolLM2-135M 冻结骨干，只训练两个工具输出行共 1152 个参数，测试集准确率从 0.50 提到 0.9375。两项都只做离线策略比较，没有用来训练线上 DeepSeek，也没有真人实验，不能说明玩家学得更好。

交付的是一个能直接玩起来的东西，里面的 AI Coach 叫小芽。游戏环境只做到足以产生真实选择的范围：训练、本地对战、养成和配招。游戏本体里唯一要动手的就是营地里的养成和配招这两块，它们决定对局里有哪些牌可打。小芽的三个角色共用同一份对局局面、同一份记忆，区别在位置和分量：军师和老师在战斗区顶部那条提示里，各说一句短话，点开才展开依据；陪练在左下角，自己找时机开口。

2.2 三个角色

三者共用上面那套机制，但说出来的东西各不一样。下面按角色分开讲。

2.2.1 军师

核心职责：帮玩家做当下的决策，但不替玩家操作。

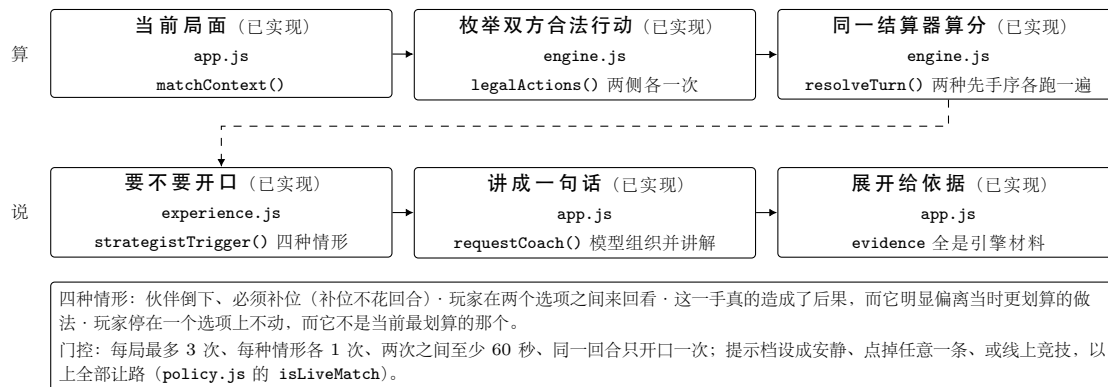


图 2 军师的机制：先把这一回合算完，再决定说不说、怎么说

军师把这一回合的取舍摆出来，玩家看到的第一行永远是一句短话，比如「这一回合优先考虑『火花』，可比较的备选是『余烬追猎』」；往下点开才是依据。算分和写话是分开的：引擎把双方这一回合所有合法行动列出来，每一种组合都放进同一个结算器里真跑一遍，算出一个分数，分数高的排前面；模型拿到这堆事实，决定这一句怎么组织、先说哪个取舍、带上哪几条依据，然后讲成玩家听得懂的话。它不参与算分，也不决定哪些行动合法。这条链见图 2，它开口时折叠的那一行见图 3。

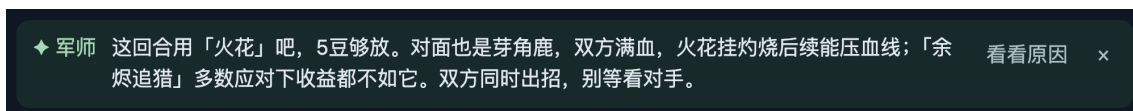


图 3 军师开口时先给一句话；点「看看原因」才展开依据，右侧「x」是本局不再打断

这个分数只用来排序，不是胜率，它只管这一回合。军师保证的是规则事实正确、行动合法，以及单回合候选排序前后一致；它不宣称得到整局的全局最优策略。双方速度相同时，谁先出手会改变这一回合的结果，所以两种先后顺序都各跑一遍，两个结果一起算进去。后续可以在现有结算器上增加有限深度的多回合模拟，把单回合评估扩展为对长期收益的评估。

军师也会主动开口，但只在四种情形下：伙伴倒下、必须补位（补位不花回合）；玩家在两个选项之间来回看；玩家这一手真的造成了后果，而它明显偏离当时更划算的做法（手上有能一击收尾的招，却没打）；玩家停在一个选项上不动，而它不是当前最划算的那个。每局最多开口 3 次，每种情形各 1 次，两次之间至少隔一分钟。把提示档设成安静（只回应提问），或者在对局里点掉任意一条，上面这些全部让路。

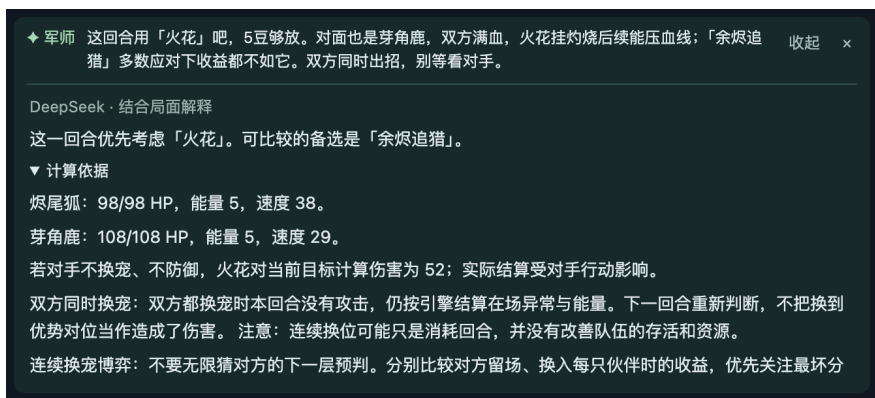


图 4 军师提示条与它展开的计算依据

图 4 是其中一条提示展开之后的样子。下面那块「计算依据」全是引擎给的材料：双方当前的血量、能量和速度，这一招打在当前目标身上算多少伤害，以及这条判断引用了哪几条知识卡——每条都带一句它自己的反例。依据里不会出现对手这一回合要出的那一手：两边锁定行动之前，军师看不到，也不允许猜。

2.2.2 陪练

核心职责：记得玩家发生过什么，并在合适时机陪伴，但不给战术答案。

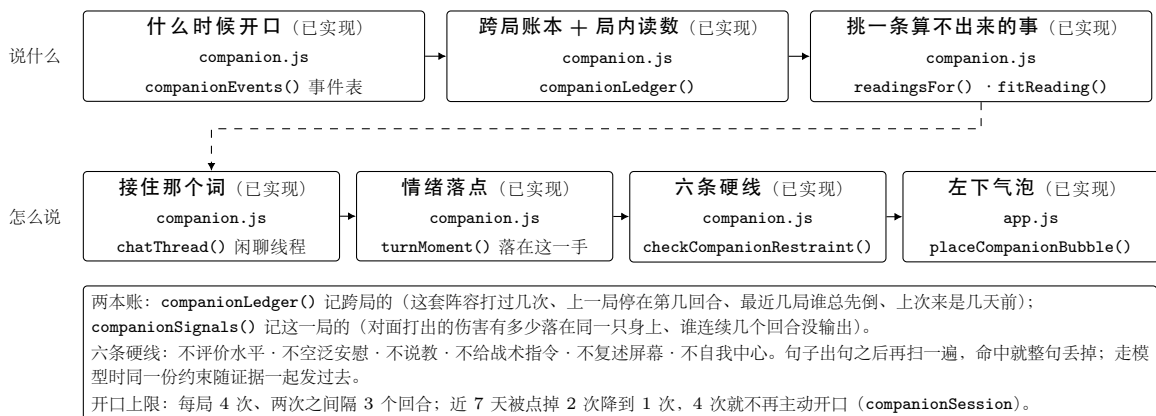


图 5 陪练的机制：从两本账里挑一件玩家自己算不出来的事，再决定怎么说

陪练是三个角色里唯一一直在场的那一个。它不等人召唤，也不是点开才出现的客服：对局进行中它会自己说话，位置在战斗页左下角，独立于聊天面板。每一条都能点掉，点掉之后这一局它就不再开口。主动开口有上限：默认每局 4 次，最近七天被关掉 2 次降到 1 次，关掉 4 次之后这一局就不再主动说话。这条链见图 5。

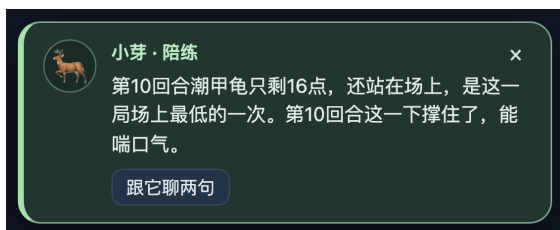


图 6 陪练气泡：不打开聊天也能看见，可以点掉

它说出口的每一句，都要落在一件玩家自己算不出来的事上。一类是跨局记录：这套阵容打过几次、上一局停在第几回合、最近几局最先倒下的总是哪一只、上次来是几天前。一类是跨回合的统计：对面这一局打出的伤害有多少落在同一只身上、某一只连续几个回合没打出输出。图 6 里那句是第二类——「是这一局场上最低的一次」这个判断，得把整局每一回合的血量都比一遍才说得出来，屏幕上的血条只显示此刻的数值。

情绪是它和另外两个角色最不一样的地方，但情绪必须有落点。它只会用五种态度：可惜、漂亮、悬、憋屈、松口气；每一种都要绑在刚结算的那一手上，说清是第几回合、哪一只、哪一记技能。比如一记收尾：「第 6 回合那记余烬追猎，把还剩 55 点血的炽鬃狮一记收掉了。整局你都没打出过这么重的一下。第 6 回合这一下，漂亮。」反过来，讲它自己的感受是不允许的：「我看得有点急」这类句子会被拦下来，它把玩家变成了来看它着急的旁观者。

它记得的事全部来自本机记录，缺一条就不说，不编。连败几局、最近赢没赢、上一局打到第几回合、还剩几瓶回复药、本命是哪只、上次玩是几天前，加上玩家自己声明过的偏好和目标。这些字段决定它用哪一档语气：平常就事论事，玩家主动问就说多层，连着输了两三局，别的话就收起来，只陪着坐一会儿，收尾那句是「到这儿也行，先歇会儿」。档位本身也写进模型看到的证据里，它说出的每一句都能回到一条本机记录上。

闲聊是它的另一个出口。玩家主动搭话时，它先接住玩家自己用过的那个词，再往下说：一句「今天有点累」，回的是「今天累了啊——那就先歇着，不用急着做什么」（图 7）。这一轮里不报战绩、不提回合数，也不问对局。这是「每句话都要说一件真事」唯一一处例外——人在说心情的时候，要的不是数据。问候和道谢也一样，先应一声：一句「你好」回的是「上午好——今天这才刚开头。慢慢来，不急。」问候句按当前时段说，这一轮同样不报战绩、不提回合数——第二句是在场，不是账本。



图 7 陪练的聊天面板：说心情那一轮，它先接住玩家自己的那个词

它的边界写得很死：不给战术指令（那是军师的事），不评价玩家打得怎么样，不说教，不空泛安慰。这几条是硬线：句子出句之后还要再扫一遍，命中就整句丢掉，走模型时同一份约束随证据一起发过去。它和军师可以同时说话：军师在顶部条上给战术，它在左下角陪着，两处位置不同，说的也是两回事。

2.2.3 老师

核心职责：解释为什么，并只看没有提示时的独立选择判断这一课要不要再教——真正学会仍属未验证。

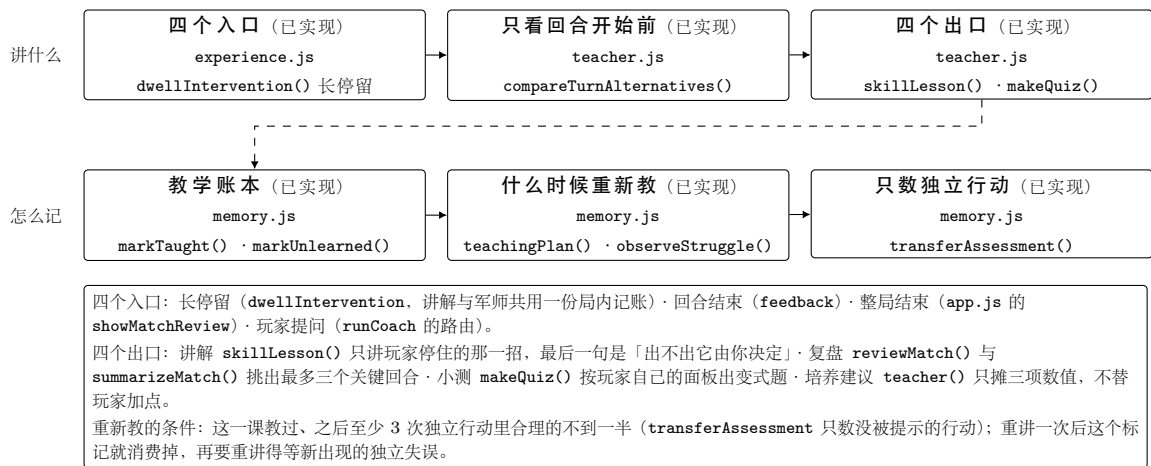


图 8 老师的机制：只用回合开始时的公开局面，讲完记进教学账本

老师的出口有四个：局内讲清玩家停在哪个技能上，复盘一个回合或一整局，按玩家自己的面板出小测，以及在营地给培养建议。这套机制见图 8。

局内它只讲玩家停住不动的那一个技能：什么属性、要花费几豆、打在当前目标身上算多少伤害、打完还剩几豆，以及它在什么条件下比手里别的选项更合适。它只解释这一招，不催出招，最后一句写着「出不出它由你决定」（图 9）。如果玩家停的那一招恰好是当前最划算的，讲解的活归老师，军师不开口——同一件事不安排两个人说。

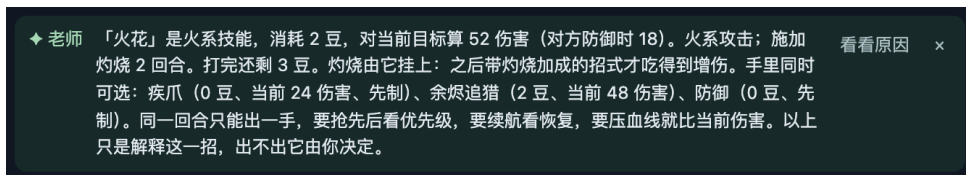


图 9 老师的单回合讲解：只解释停住的那一招，不替你决定出不出

整局结束的复盘是它最重的一块：上面一行是这局的结论，展开区里是按减员和生命变化挑出来的最多三个关键回合（图 10）。每个回合写清当时双方还剩多少血、这一手实际打出了什么，以及当时还能选哪些做法、它们比实际这一手好多少。整局统计只讲发生过的行动：没换过宠、没用过道具就不提。

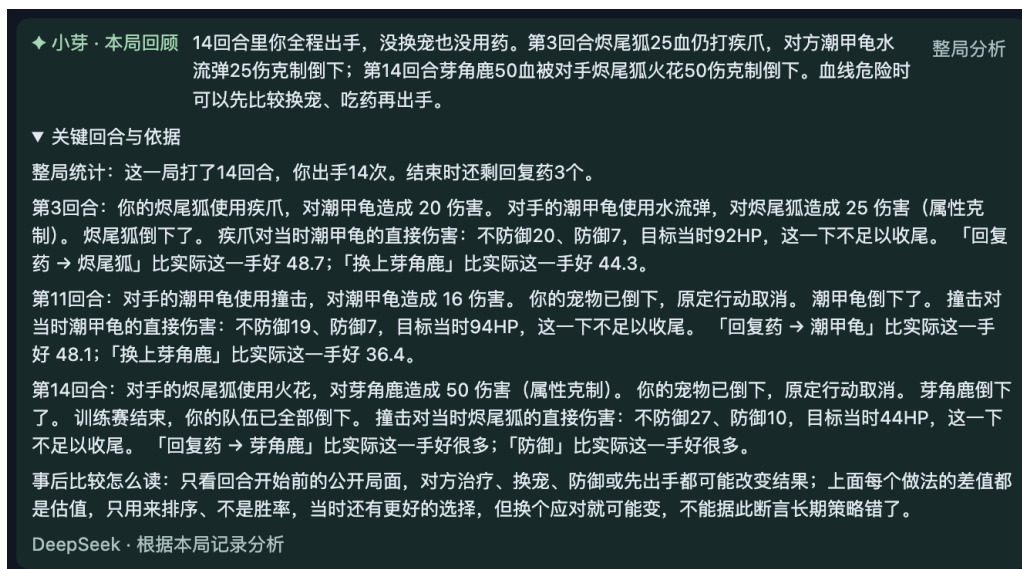


图 10 老师的整局复盘与挑出来的关键回合

复盘有一条硬口径：一律只看回合开始前的公开局面，不用对手实际打出的那一招。否则任何一次失败都能被解释成「你当时选错了」。把对手那一步换掉，复盘的结论不变，这一条有测试守着。

小测按玩家自己的面板出变式题，不背题库。面板速度 38，培养一次敏捷是 41，对手速度 41，答案就是「无法确定」——同速时谁先出手由规则里的随机过程决定，不能保证先手；选项里专门留着「不确定」这一项（图 11）。

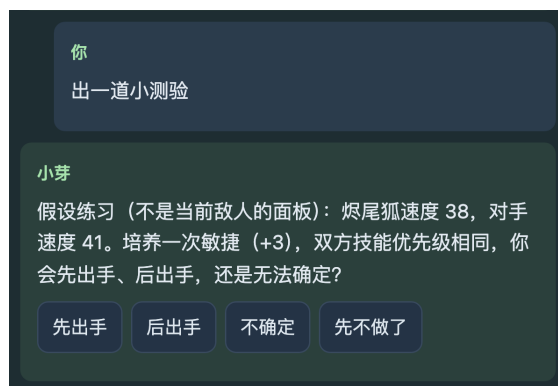


图 11 小测按当前面板出题：这一道的答案只能是「不确定」

培养建议是它唯一涉及成长的入口：把三项数值加一点之后的对照摊开——生命 98 到 110、攻击 27 到 31、速度 38 到 41（图 12）——并说明这一次加点会不会改变与当前关卡首发的先后关系。加点这件事由玩家自己按，老师只给数。



项目	现在	加1点后
生命	98	110
攻击	27	31
速度	38	41

图 12 培养建议：三项数值加一点之后的对照

真正花心思的是教学账本，它记三件事：讲过哪一课、哪一课需要重新教学、这一课被提示过和没被提示过的行动各有几个。讲过又没有新的反证，就不重复讲。要不要重新教学，只看没有提示时的独立选择：只有当军师在局内独立看到三次以上同类失误——至少 3 次独立行动、其中合理率不到一半——这一课才会被标回需要重新教学，它也说得出为什么：「这一课教过，但之后 4 次独立行动里有 3 次不合理」。重新讲过之后这个标记就消费掉，再要它重讲，得等新出现的独立失误。

教学账本只数没被提示过的行动，理由很简单：被提示后做对，不等于没人提醒时也会做。相关教育研究也讨论过类似问题¹——提示会把学生推向高效步骤，用过的学生反而被判成「不需要帮助」。小芽这里是同一件事：判断玩家有没有掌握一个知识点，只看他在没有被提示的时候做出的选择。这套账本是自建的计数与规则，不是心理测量模型：它只回答这一课教过没有、这一次要不要再教，不估计掌握概率。

3 设计与实现中的关键难点

这一章讲这套东西难在哪、我在哪几个地方差点做错，以及我是怎么发现的。

3.1 为什么游戏事实不能让模型自己算

一局对战里，玩家看到的每一个数都得对得上：这一招打在当前目标身上算多少伤害、打完还剩几豆、这个回合有哪些行动是合法的、伙伴倒下之后补位换谁上来，以及速度相同时谁先出手。这几件事互相咬在一起，一处数字错，后面所有解释都跟着错——伤害少算了 5 点，本来能收尾的一手会被讲成收不掉，玩家照着这句话去做，输的是这一局。

所以伤害、合法行动、能量、换宠规则这些都由规则引擎当场算出来，模型拿到的是算完的确定结果，它只在这些核验过的信息上理解和解释。提示词里写一句「不要算错」不能代替程序计算与校验：模型看不到随机数、也看不到对手锁定的那一手，它算不了；就算它算得出来，它给的答案也没法被逐条核验。基准是这个：凡是能用代码确定下来的事实，就不交给模型猜。

¹K. R. Koedinger & V. Aleven, *Exploring the Assistance Dilemma in Experiments with Cognitive Tutors*, Educational Psychology Review, 2007. 论文里被讨论的正是「给不给提示」这个两难。

3.2 为什么复盘不能使用后来才知道的信息

老师评价某一回合，只能用玩家做决定之前已经公开的信息。这一条比我原先想的重要：一局打完，双方的记录都在手上，我随时可以让老师知道对手当时出的是什么；一旦允许它用这条信息，任何一次失败都能被解释成「你当时选错了」——那句话在后来的结果面前永远说得通，但它在玩家做决定的那一刻并不成立，等于拿结果去倒推当时该不该选。

实现上，每个回合都留了一份回合开始时的公开局面，复盘只看这一份，不看对手实际打出的那一招。这条口径有一条测试守着，测试的思路是替换：把对手后来实际做的动作换成别的，复盘的结论不应该跟着变。它变一次，就说明有信息从未来漏了进来。

3.3 为什么主动 Coach 不能一直说话

真正的问题不是「每次生成什么」，而是「什么时候根本不该出现」。军师和陪练都是会自己开口的角色，写起来最容易只顾着把话说好，结果每一回合都有话可说，玩家几分钟内就被打断十几次。让一个提醒变得可用的，全是限制它出现的机制：每局上限、两次之间至少隔多久、玩家点掉之后这一局不再开口、提示档设成安静时只回应提问；最近七天被关掉过的次数还会往下压额度，关得多了就不再主动开口。老师的长停留讲解和军师共用同一份额度，同一局里两个人加起来也不会没完没了。

这些限制没法靠读代码判断合不合理，只能真的打一局：我在同一个局面里连着收到几条提示，才知道额度定得太宽。所以我也做了一个离线实验，看提醒时机本身值不值得学：在一个模拟环境里，学到的时机比「到点就提醒」省下大约三分之二的打扰，代价是给出的帮助量也从每次 1.38 降到 0.86，省打扰和给帮助是一笔要算清的交换。这个实验是离线探索，只用来比较策略，现在线上的开口时机仍然由上面这些固定规则决定。

3.4 测试通过，为什么功能仍可能有问题

`npm test` 全绿不等于功能可用。踩过的坑分成三类，每一类都让某次改动顺利通过了全部检查。

第一类是函数正确，但没有接到产品路径上。新增的对局规则模块没进静态资源白名单，磁盘上的白名单、测试、模块图都是对的，页面却报 404，一行 JS 都没有执行，我看到的是一片空白。陪练的主动气泡模板写好了，调用它的那个函数在任何地方都没有被调用过，所以气泡从不出现，而它自己的测试条条通过——因为没有一条测试问「谁调用它」。这两条都不是读代码读出来的，只有我把页面打开、真打一局才看得见。

第二类是程序在跑，但量的不是原本想量的东西。玩家补位的那一回合本来就不该出招，却被计成一次「引擎兜底」。犹豫判定量的是「距第一次停留的经过时间」：我在火花上停了一下，然后移去对方面板操作了 13 秒，回来碰「防御」，界面就弹出「你在火花和防御之间停留了 13 秒」——那 13 秒我根本没在看这两个选项。信号要量的是花在这上面的时间，不是经过了多少时间。

第三类是检查本身有问题，错误也能通过。冒烟测试里的伙伴卡数量写死过，宠物增加之后那个数字不再对得上；语义检索的语料是手工维护的第三份副本，卡片改了它

不知道；工具参数曾经用合法行动列表的下标，而索引会随能量、道具和倒下整体位移，同一个下标在不同回合指向不同行动；禁止内部术语进玩家文案的那条检查有盲区，取局内提示时没有传事件参数，那条理由的正文根本不在覆盖范围内，把它换成错话也照样通过。

所以每写完一条检查，我会先做一次反向验证：把错误的东西故意放进去，看它会不会失败。检查不失败，就说明它测的不是我以为的那件事。期望值也从被测对象自己推导，不再手写常量。

4 交付结果

三种角色已经在实际游戏流程中形成可运行路径，模型不可用时也能退回本地结果。下面不再重复功能，而只看这些机制经过了什么验证，以及这些结果能说明到什么程度。

表 1 是这五组问题各自用的方法和跑出来的数，表 2 是同一批结果能说明什么、不能说明什么。其中「主动提醒」和「小模型」两行是那两个离线策略实验，用来比较做法，不在线上主链。

表 1 每组问题用的方法和跑出来的数

问题	方法	当前结果
检索	该用哪条路：四臂对照（不检索 / 词项 / 混合 / 反例加条件），同一份 128 条查询	前 3 命中率 0 / 0.884 / 0.902 / 0.911；112 条有正解的查询里命中正确卡的 0 / 99 / 101 / 102 条
受控工具调用	模型真的会连查几次：44 条真实模型记录，逐条统计调用次数与停止方式	调用 0 次 36 条、1 次 8 条；1 条参数非法、1 条重复被拦；没有一次形成多步工具链
主动提醒	该在什么时候开口：离线策略对照（不提醒 / 固定规则 / 随机初始化 / Q-learning），并做了一档行为分布变化的对照	奖励 0 / -1.113 / -0.770 / 1.116（分布变化时 0 / -2.33 / -1.679 / 0.581）
小模型	能不能被训练到会选工具：SmolLM2-135M 冻结骨干，只训两个工具输出行共 1152 个参数，24 训练 / 8 验证 / 16 测试	未训练 16 条全选同一工具，准确率 0.50；训练后 0.9375，三种子一致
回归	改完有没有碰坏别的：npm test 全量跑（单元测试并行、浏览器测试串行）；无头浏览器冒烟：从营地选人、出招打到结算	366 项全通过，0 失败；冒烟走完整局无报错

表 2 同一批结果能说明什么、不能说明什么

问题	能说明什么	不能说明什么
检索	词项检索确实比不检索强得多：前 3 命中率 0.884，不检索是 0；反例加条件这一臂数值最高，比词项高 2.68 个百分点	这 2.68 个百分点落在噪声范围内：128 条里只有 7 条两臂结果不同，其中 5 条只有反例臂命中、2 条只有词项臂命中，差距分不出来；量的只是「正确卡进没进前 3」，说明排序有改善，说明不了最终回答质量
受控工具调用	被指定的那一步在真实调用里都执行成功，参数校验与重复拦截也真的触发过	没有观察到模型自己发起的多步工具链，上限 3 次这件事只有确定性测试覆盖
主动提醒	在模拟环境里，学到的时机比「到点就提醒」省打扰；但提示次数下降的同时帮助量也下降，说明减少打扰和提供帮助之间存在取舍	奖励函数和我自己写的模拟器就是全部环境，没有真人；省打扰的同时帮助量也在下降
小模型	在这个二选一的小任务上，这 1152 个参数确实学到了东西	手写的二选一场景，只是小模型工具选择实验，不是完整的多步工具链训练，也没微调线上 DeepSeek
回归	已知行为没被改坏，玩家会走的那条路是通的	自动测试通过不代表真人有效；测试只数它被要求数的东西，框架本身也出过错（见第 3 章）

上面两组离线数字都取自单次固定的随机种子（干预时机那组是 seed 71），不是多种子平均；44 条真实模型记录生成于 9 月 17 日，早于 9 月 18 日那次「谁决定第一个工具」的逻辑改动，所以那批记录的调用次数分布只对应生成它时的代码版本。

模型没接上、超时、输出过长，或者和工具回执对不上时，规则建议、复盘、培养建议和小测仍由本机引擎给，屏幕上显示本机已核验的结论，并说明为什么退回。还没做到的地方：语音没做，联网对战是同机分屏，仓库里的演示是无头浏览器抓的截图，不是真人录屏，也不是用户测试；玩家是不是真的学会了，要等真人用一遍、把提示撤掉之后还能自己做对，才算数。

5 运行与使用

代码在 <https://github.com/SeRendizc/pet-game-coach>。核心 Demo 使用 Node.js 20+，没有第三方 npm 运行依赖；语义检索和离线模型实验另有 Python 与模型依赖。启动就是一条命令：`npm start`，然后浏览器打开 <http://127.0.0.1:8765/> 就能玩。

不接模型也能完整游玩，规则建议、复盘和小测都由本机引擎给。想接模型，就打开 <http://127.0.0.1:8765/connect.html> 填上 DeepSeek API Key，密钥只留在后端进程内存里；也可以先跑一次 `./scripts/start.sh --save-key` 存进钥匙串。想看某个能力：营地是培养配招，出征选关卡与伙伴，对局里提示条在战斗区顶部、陪练在左下角，

复盘和加点对照在局后与培养页。

开发中使用 AI agent 辅助代码实现、单元测试与文档整理；需求理解、系统架构、角色边界与最终验收由我负责——功能有没有真的生效，我只看自己把页面打开、真打一局的结果。